Технология распределенных реестров

Технология распределенных реестров – это подход к обмену и хранению информации, при котором:

- каждый участник может обладать полноценной копией реестра
- синхронизация копий реестра происходит на основе протокола достижения распределенного консенсуса, то есть соглашения среди участников на добавление новой информации
- каждый участник взаимодействия может иметь доступ к истории транзакций

Такой подход позволяет обеспечить единый источник достоверных данных без необходимости централизованного доверенного посредника.

Блокчейн

Одной из наиболее известных реализаций распределенных реестров является блокчейн.

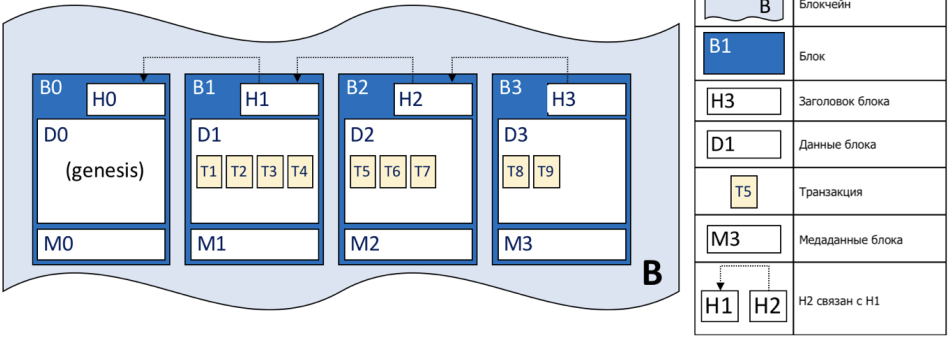
В блокчейне информация группируется в цблоки, которые связаны между собой в цепочку. Каждый новый блок содержит ссылку на предыдущий.

Это обеспечивает:

- целостность данных
- невозможность изменения истории
- прозрачность всех транзакций

Термин «**блокчейн**» иногда заменяет название всей технологии, потому что такой способ записи используется чаще других.

Структура блокчейна



Типы распределенных реестров

По уровню доступа участников сети распределенные реестры можно разделить на несколько типов.

Публичные сети

Любой участник может подключиться к сети и участвовать в ее работе.

Частные сети

Доступ к сети ограничен, участники известны и контролируются.

Гибридные сети

Часть данных является публичной, а часть доступна только определенным участникам.

Для корпоративных систем чаще всего используются частные или гибридные сети.

Основные свойства распределенных реестров

Технология распределенных реестров обладает рядом ключевых характеристик:

- децентрализация — данные хранятся на множестве независимых узлов в P2P-сети
- неизменяемость — записанные данные невозможно удалить или изменить
- консенсус — все узлы сети используют единый алгоритм для синхронизации и подтверждения правильности обновлений реестра
- прозрачность — участники имеют доступ к истории операций
- криптографическая безопасность — используются цифровые подписи
- автоматизация — возможна реализация смарт-контрактов

Смарт-контракты

Смарт-контракт — это программа, хранящаяся в распределенном реестре и автоматически исполняющая условия договора.

Исполнение смарт-контракта происходит при наступлении определенных условий.

Например, в системе эксплуатации дронов смарт-контракт может проверять:

- наличие действующей сертификации дрона
- наличие страхового полиса
- наличие разрешения на выполнение полета

Если все условия выполняются, система может автоматически разрешить выполнение миссии.

Платформы распределенных реестров

Существует множество платформ для реализации распределенных реестров.

Наиболее известные из них:

- Ethereum — публичная блокчейн-платформа со смарт-контрактами на языке Solidity
- Tendermint / Cosmos — модульная платформа для создания собственных блокчейн-сетей
- Hyperledger Fabric — корпоративная платформа распределенного реестра

Для корпоративных систем с известными участниками наиболее подходящей является платформа Hyperledger Fabric.

Почему выбран Hyperledger Fabric

Hyperledger Fabric изначально проектировался для использования в корпоративной среде.

Основные преимущества:

- идентификация участников сети
- контроль доступа к данным
- высокая пропускная способность
- низкая задержка подтверждения транзакций
- возможность разделения данных между организациями

Эти характеристики делают платформу подходящей для построения инфраструктуры взаимодействия участников системы эксплуатации дронов.

Архитектура Hyperledger Fabric

Основными компонентами сети Hyperledger Fabric являются:

- Peer nodes — узлы сети, хранящие копии реестра
- Ordering Service — сервис упорядочивания транзакций
- Chaincode — смарт-контракты
- Certificate Authority — центр сертификации участников
- Channels — изолированные каналы обмена данными

Каждая организация в сети может иметь собственные узлы и участвовать в консенсусе.

Типы узлов в сети

В сети Hyperledger Fabric используются несколько типов узлов.

Endorsing Peer

Узел, который симулирует исполнение транзакции и возвращает результаты выполнения клиентскому приложению вместе со своей подписью.

Ordering Service

Распределенный сервис на нескольких узлах, служит для формирования новых блоков распределенного реестра и создания очередности исполнения транзакций.

Committing Peer

Узел, на котором происходит проверка транзакций на соответствие политике одобрения, установленной для конкретного приложения, перед их записью в реестр.

Центр сертификации

Для идентификации участников используется центр сертификации (Certificate Authority).

Он выполняет следующие функции:

- выдача цифровых сертификатов участникам сети
- подтверждение принадлежности участников к организациям
- обеспечение аутентификации пользователей и узлов сети

Сертификаты основаны на стандарте X.509 и инфраструктуре открытых ключей (PKI).

Модель выполнения транзакций

В большинстве блокчейн-платформ используется модель:

Order $\rightarrow$ Execute

Hyperledger Fabric использует модель:

Execute $\rightarrow$ Order $\rightarrow$ Validate

1. выполнение транзакции и ее одобрение
2. упорядочивание транзакций
3. проверка транзакций и запись в реестр

Такая архитектура позволяет повысить масштабируемость и производительность сети.

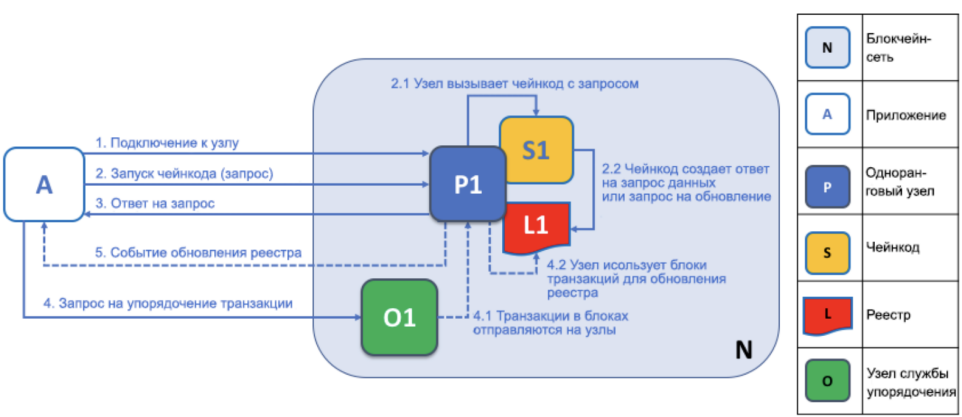
Взаимодействие приложений с сетью

Приложения взаимодействуют с сетью Hyperledger Fabric через SDK.

Основные этапы выполнения транзакции:

1. приложение формирует транзакцию
2. транзакция отправляется endorsing peer
3. выполняется смарт-контракт
4. ordering service формирует блок транзакций
5. committing peers проверяют и записывают блок в реестр

Взаимодействие приложений с сетью



Каналы

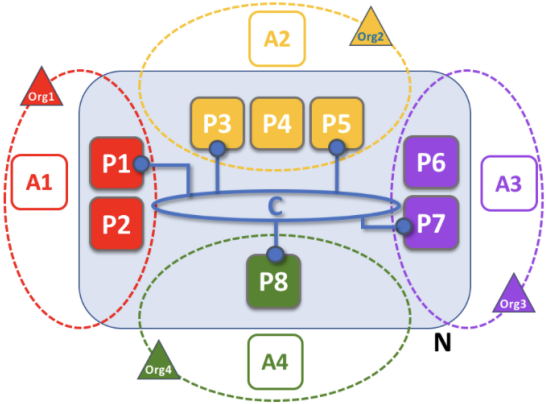
Канал представляет собой изолированную подсеть внутри блокчейн-сети.

Каналы позволяют:

- разделять данные между организациями
- обеспечивать конфиденциальность транзакций
- реализовывать независимые бизнес-процессы

Каждый канал имеет собственный реестр и набор смарт-контрактов.

Архитектура сети



N	Блокчейн-сеть	L	Реестр
C	Канал	A	Приложение
P	Одноранговый узел	PA, C	Доверитель PA (например, A1, P5) связывается по каналу C.
		Org	Организация
A1, R, P1, P2	Одноранговые узлы P1, P2 и приложение A1 принадлежат организации R		

Смарт-контракт сертификации прошивок

В системе реализован отдельный смарт-контракт **FirmwareContract**, который отвечает за управление сертифицированными прошивками дронов.

```go
type FirmwareContract struct {
  contractapi.Contract
}
```

Контракт поддерживает следующие группы операций:

- **создание записи:** `CertifyFirmware`;
- **чтение данных:** `ReadFirmware`, `ListFirmwares`;
- **обновление:** `UpdateFirmware`;
- **отзыв сертификации:** `RevokeFirmwareCertification`.

Структура объекта Firmware

Основная сущность контракта — объект **Firmware**, содержащий:

```
type Firmware struct {
    ID                  string    `json:"id"`
    SecurityObjectives  []string  `json:"security_objectives"`
    SoftwareObjectives  []string  `json:"software_objectives"`
    CertifiedAt         string    `json:"certified_at"`
    CertifiedBy         string    `json:"certified_by"`
}
```

Пример реализации FirmwareContract

```go
func (c *FirmwareContract) CertifyFirmware(...) error {
    if err := assertRole(...); err != nil {
        return err
    }

    certifierID, _ := cid.GetID(ctx.GetStub())

    firmware := Firmware{...}

    key, err :=  makeFirmwareKey(ctx, id)
    if err != nil {
        return err
    }

    return putState(ctx, key, firmware)
}
```

Логика работы контракта FirmwareContract

Функция `CertifyFirmware` реализует создание новой записи о сертифицированной прошивке в распределённом реестре.

При выполнении транзакции контракт:

1. проверяет роль вызывающей стороны;
2. получает идентификатор участника из контекста Fabric;
3. формирует объект `Firmware`;
4. создает составной ключ для записи в world state;
5. сохраняет объект в распределённом реестре.

DronePropertiesContract

Смарт-контракт для управления паспортами дронов и страховками.

```go
type DronePropertiesContract struct {
  contractapi.Contract
}
```

Типы данных:

- TypeCertificate — сертификаты типа
- DronePass — паспорта дронов
- InsuranceRecord — страховки

Операции:

- Выпуск сертификатов типа
- Создание/чтение/обновление паспортов
- Управление страховками

Смарт-контракт управления заказами

Смарт-контракт **OrderContract** управляет заказами на полеты дронов.

```go
type OrderContract struct {
  contractapi.Contract
}
```

Типы данных:

- Order — заказ с распределением платежей
- MissionInsurance — страховка миссии
- FlightPermission — разрешение ОрВД
- RestrictionViolation — нарушения

Жизненный цикл заказа: Создание → Назначение → Одобрение → Подтверждение → Полёт → Завершение → Оплата